# Pre-training, Fine-tuning, and Prompting Strategies for Java Bug Fixing

Aidan Basloe
CSCI 555: GenAI for SD

April 30, 2026

**Abstract**

This report presents the implementation and results of three assignment pipelines for Java bug fixing: a pre-train-then-fine-tune T5 pipeline (Pipeline A), a fine-tune-only T5 pipeline that skips pre-training (Pipeline B), and a prompting pipeline based on Qwen with and without retrieval augmentation. The project uses one shared SentencePiece tokenizer trained on a pre-training corpus of 50,000 Java methods and reuses that same tokenizer for all later stages so that any differences between Pipeline A and Pipeline B come from pre-training rather than tokenization. The manually configured T5 model uses a T5-small style encoder-decoder architecture with a 16,384-token vocabulary learned from the project corpus. Pipeline A performs span-corruption pre-training on 48,055 filtered methods and then fine-tunes on the Java bug-fixing task, while Pipeline B fine-tunes the same architecture directly from random initialization using identical hyperparameters. The final evaluation compares all four reported settings on the same 6,545-example test split using exact match and CodeBLEU. The strongest overall result came from Qwen with retrieval-augmented prompting, while among the two small-model training pipelines, the fine-tune-only baseline substantially outperformed the pre-trained model in this run.

## 1 Introduction

The goal of this assignment is to compare multiple ways of solving Java bug fixing with the same downstream evaluation. The project asks for two small-model training pipelines and one prompting-based comparison. Pipeline A trains a shared tokenizer, pre-trains a T5 model on unlabeled Java methods, and then fine-tunes that model on bug fixing. Pipeline B uses the same tokenizer and the same T5 architecture but skips pre-training and fine-tunes directly from random initialization. The final comparison adds a larger instruction model,

Qwen, first in zero-shot mode and then with retrieval-augmented prompting using bug-fix examples from the training set.

The final implementation is contained in `assignment_pipelines_a_b_combined.ipynb`. The notebook is organized as a complete end-to-end workflow: configuration, tokenizer training, model construction, Pipeline A pre-training, shared fine-tuning helpers, Pipeline A and Pipeline B fine-tuning, Qwen-based retrieval and prompting, and final four-way evaluation. The report below follows that same structure so the discussion stays tied to the exact implementation used for the final run.

# 2 Tokenizer Training and Model Configuration

## 2.1 Tokenizer Training

The notebook first creates one shared SentencePiece tokenizer and saves it under `outputs/tokenizer/sp_c`
The pre-training text comes from 50,000 sampled Java methods taken from the CodeSearchNet Java training split and written to `outputs/corpus_java_50k.txt`. In the tokenizer stage, the notebook either loads the cached corpus and tokenizer or regenerates them if the force flags are enabled.

The tokenizer is trained with the Unigram algorithm and a target vocabulary size of 16,384. The implementation explicitly sets `pad_id=0`, `eos_id=1`, `unk_id=2`, and `bos_id=-1` (disabled, since T5 does not use a beginning-of-sequence token during decoding; `decoder_start_token_id` is set to the pad ID instead). It also includes the 100 sentinel tokens `<extra_id_0>` through `<extra_id_99>` through `user_defined_symbols`. This part is important because the same sentinel tokens are later used by the span-corruption objective in Pipeline A. After loading the tokenizer, the notebook performs several sanity checks: it prints the actual vocabulary size, confirms the pad/eos/unk IDs, checks that the first sentinel tokens have valid IDs, and shows a sample tokenization of a Java method signature and return statement. In the final run, those sanity checks showed a vocabulary size of 16,384, special-token IDs of 0/1/2, and valid sentinel IDs beginning at 3.

The most important design choice in this section is that the tokenizer is trained exactly once and then reused everywhere else in the notebook. Both fine-tuning pipelines read and write through the same SentencePiece model, and the Qwen comparison uses the same task data split for evaluation. This directly enforces the assignment rule that tokenizer differences should not confound the comparison between pre-training and no pre-training.

## 2.2 Model Configuration

The core neural model is built by the notebook function `build_t5_small_model()`. Instead of borrowing a pre-trained checkpoint shape, the notebook defines the configuration

directly in code through `T5Config`. The main parameters are `d_model=512`, `d_ff=2048`, `d_kv=64`, `num_heads=8`, `num_layers=6`, and `num_decoder_layers=6`. The tokenizer-derived special tokens are inserted into the configuration with `decoder_start_token_id = PAD_ID`, `eos_token_id = EOS_ID`, `bos_token_id = PAD_ID`, and `pad_token_id = PAD_ID`. The notebook then calls `model.resize_token_embeddings(VOCAB_ACTUAL)` so the embedding matrix is resized to match the 16,384-token SentencePiece vocabulary. This call is required after any `T5Config` manual construction because the default embedding table will not match a custom vocabulary without it.

A probe model created from this builder reported 52.4 million parameters. The slight difference from the canonical T5-small figure of ∼60M arises because the embedding layer is sized to the project's 16,384-token vocabulary rather than the 32,128-token vocabulary used in the public T5-small release; with a smaller vocabulary, the embedding matrix contributes fewer parameters. The helper function `configure_special_tokens()` is then used throughout the notebook to reapply these tokenizer-specific settings whenever a model is loaded from disk. This keeps the tokenizer-model contract consistent across pre-training, fine-tuning, generation, and evaluation.

# 3 Core Model, Pre-training, and the Two Fine-tuning Pipelines

## 3.1 Pipeline A Pre-training

Pipeline A begins with an unlabeled Java corpus. The notebook encodes each method with SentencePiece, truncates to a maximum length of 512 tokens, and filters out methods shorter than 10 tokens or longer than 512 tokens. Although the raw pre-training sample size is 50,000 methods, the final cached tokenized corpus used for pre-training contains 48,055 examples after filtering.

The pre-training objective is implemented in `corrupt_tokens()`. For each tokenized method, the notebook masks 15% of positions, groups consecutive masked positions into spans, replaces each span in the input with a sentinel token, and constructs the target sequence from the removed spans prefixed by their corresponding sentinel tokens. The `SpanCorruptionDataset` class applies this corruption dynamically inside `__getitem__()`, so a method can receive a fresh corruption pattern whenever it is sampled. The `collate_pretrain()` function pads the corrupted inputs, builds an attention mask, and pads labels with `-100` so loss is computed only on valid target positions.

The pre-training loop is implemented in `run_pipeline_a_pretraining()`. It uses `AdamW` with a learning rate of $5 \times 10^{-4}$, a batch size of 8, three epochs, random seeding for deterministic shuffling, and checkpoint support so interrupted runs can resume. Table 1

reports the training loss at the end of each epoch, confirming that the span-corruption objective converged steadily over the three passes. Although the code supports step checkpoints for recovery, the final saved product for the assignment is still the final pre-trained model stored in `outputs/pipeline_a_pretrained_model`.

| Epoch | Training Loss |
|---|---|
| 1 | 3.4521 |
| 2 | 2.8307 |
| 3 | 2.5194 |

Table 1: Pipeline A span-corruption pre-training loss per epoch on 48,055 filtered Java methods.

## 3.2 Shared Fine-tuning Setup

Both fine-tuning pipelines use the same bug-fixing dataset cache built from `google/code_x_glue_cc_code_` with the `medium` configuration. The notebook tokenizes the `buggy` field as the input and the `fixed` field as the target, appending EOS to both sequences and storing the result in `outputs/code_refinement_medium_tokenized.pt`. The cached split sizes are 52,364 training examples, 6,546 validation examples, and 6,545 test examples. The test split was held out and used only once, for final reporting; all checkpoint selection and hyperparameter decisions were made on the validation split.

The fine-tuning hyperparameters are also shared and written to `outputs/fine_tuning_hyperparam` In the final run, both pipelines use 3 epochs, batch size 8, evaluation batch size 8, learning rate $5 \times 10^{-5}$, weight decay 0.01, gradient clipping at 1.0, generation maximum length 512, and beam search with 4 beams. The `CodeRefinementDataset` and `collate_refinement()` helpers are reused across both pipelines. Validation loss is computed after each epoch, the best-validation checkpoint is saved separately from the final checkpoint, and generation on the test set uses the same decoding settings for both models.

## 3.3 Pipeline A vs. Pipeline B

The only architectural difference between the two fine-tuning pipelines is the initialization source. Pipeline A starts from `outputs/pipeline_a_pretrained_model`, which is the result of span-corruption pre-training on Java methods. Pipeline B starts from a fresh call to `build_t5_small_model()` and therefore begins with random weights. Everything else stays matched: the same tokenizer, the same input and target encoding, the same training split, the same optimizer settings, the same validation procedure, and the same generation settings.

This setup creates a direct test of whether the pre-training stage helped this small model learn the downstream bug-fixing task more effectively. Because the tokenizer and

hyperparameters are locked, the observed difference in final metrics can be attributed to the initialization strategy rather than to any other experimental change.

# 4 RAG Setup and Implementation

The notebook implements the prompting comparison in a separate RAG section that uses `Qwen/Qwen2.5-Coder-1.5B-Instruct` as the generator. The retriever is built from `microsoft/codebert-b` CodeBERT was chosen over Code2Vec with FAISS (as demonstrated in the RAG lab) for two reasons. First, CodeBERT is a transformer encoder pre-trained on code–documentation pairs across multiple languages, so its contextual representations capture semantic similarity between buggy methods more reliably than Code2Vec's graph-based token aggregation. Second, at the scale of this project (52,364 training embeddings of size 768), brute-force matrix multiplication with `torch.topk` is fast enough that the added complexity of a FAISS index is unnecessary; a FAISS index would become preferable only at substantially larger retrieval corpora.

The function `embed_codebert_texts()` tokenizes buggy Java methods, obtains contextual token representations, mean-pools them with the attention mask, and L2-normalizes the resulting vectors. Mean-pooling with the attention mask is a standard and lightweight aggregation strategy that suppresses the contribution of padding tokens while preserving the full sequence context. The training split produces 52,364 buggy-method embeddings of size 768, and the test split produces 6,545 query embeddings with the same dimension.

Retrieval is performed by matrix multiplication between normalized test and training embeddings followed by `torch.topk`. The notebook stores the top-3 retrieved training indices for each test query in `outputs/rag/codebert_topk_train_indices.pt`. For the final run, each test example therefore receives three retrieved bug-fix examples from the training set.

Prompt construction is handled through two functions. `qwen_messages_zero_shot()` creates a direct instruction asking Qwen to fix one buggy Java method and return only the fixed method. `qwen_messages_rag()` builds a few-shot prompt containing three retrieved `buggy -> fixed` examples followed by the new buggy method. Both settings use the same system behavior: return only the fixed Java method with no explanation. This choice keeps evaluation cleaner because the output is expected to be executable code rather than a natural-language answer.

The generation code uses greedy decoding with `do_sample=False` and writes predictions incrementally to JSONL files so generation can resume if interrupted. The notebook then scores both the zero-shot and RAG predictions on the same 6,545-example test split used for Pipeline A and Pipeline B.

# 5 Evaluation Results and Comparative Analysis

## 5.1 Final Results

Table 2 summarizes the four final evaluation settings reported by the notebook. All four systems were evaluated on the same held-out 6,545-example test split, which was not used at any point during training or checkpoint selection.

| Stage | Exact Match | CodeBLEU |
|---|---|---|
| Pipeline A | 0.0000 | 0.0628 |
| Pipeline B | 0.0006 | 0.4704 |
| Qwen zero-shot | 0.0000 | 0.3793 |
| Qwen RAG 3-shot | 0.0058 | 0.6767 |

Table 2: Final evaluation results on the shared 6,545-example test split.

The Pipeline A metrics are especially important because they directly test whether the pre-training stage helped. In this run, Pipeline A achieved an exact match accuracy of 0.0000 and a CodeBLEU score of 0.0628. Pipeline B, which used the same tokenizer and architecture but skipped pre-training, achieved 0.0006 exact match and 0.4704 CodeBLEU. The delta reported by the notebook is therefore $-0.4076$ CodeBLEU and $-0.0006$ exact match for Pipeline A relative to Pipeline B. In other words, pre-training did not help in this experiment; it hurt downstream performance substantially. The near-zero exact match scores for both T5 pipelines reflect the difficulty of the task at this model scale: CodeBLEU at 0.47 for Pipeline B indicates that the model is generating syntactically plausible and partially overlapping output, but the fixes are rarely character-for-character identical to the reference.

The prompting results show a different pattern. Qwen zero-shot achieved 0.3793 CodeBLEU, which is lower than Pipeline B but far above Pipeline A. When retrieval was added, Qwen RAG 3-shot achieved the best overall performance with 0.0058 exact match and 0.6767 CodeBLEU. The gap between Qwen zero-shot and Qwen RAG shows that retrieval was useful in this setup. The RAG prompt provided concrete examples of how similar buggy methods were repaired, and the generator benefited from those examples when producing the final fix.

## 5.2 Analysis

The main experimental surprise is that Pipeline B outperformed Pipeline A. Since both pipelines use the same SentencePiece tokenizer and the same fine-tuning setup, the pre-training stage is the most plausible source of the difference. One interpretation is that the

small model did not receive enough useful signal from only three epochs of span-corruption pre-training on 48,055 filtered Java methods. Another possibility is that the pre-training corpus and objective did not align closely enough with the downstream bug-fixing task. Span corruption teaches the model to reconstruct masked spans, but bug fixing requires much more targeted transformations conditioned on semantic errors. In this run, the downstream fine-tuning process may simply have been more effective when the model started from a fresh random state rather than from weights shaped by a relatively small unsupervised corpus.

The comparison with Qwen also highlights a paradigm difference. The T5 pipelines require up-front training and checkpoint management, but inference is lightweight once the model is trained. In contrast, Qwen avoids task-specific training but pays for a larger generator at inference time. The zero-shot result suggests that a strong general-purpose code model already captures useful bug-fixing behavior, while the RAG result shows that retrieval can further improve performance by narrowing the generator's attention to similar examples from the task distribution.

From a deployment perspective, the results suggest different choices for different constraints. A team shipping a linter plugin or IDE extension would favor Pipeline B: a 52M-parameter model runs in milliseconds per sample on a single GPU, requires no network calls after the initial training pass, and can be bundled with the tool. For such a team, the up-front fine-tuning cost is paid once, and the task is stable enough that retraining is rare. In contrast, a research group running a one-off study on a new bug class would favor Qwen with RAG: swapping the knowledge base with new bug-fix pairs requires no retraining at all, and the system adapts immediately. The trade-off is that every inference call requires a 1.5B-parameter forward pass plus a retrieval step, which is impractical for high-throughput production use but entirely reasonable for occasional evaluation runs. Pipeline A demonstrates the intended pre-training workflow correctly, but its final metrics indicate that pre-training was not a free improvement in this particular setup and would require a larger corpus or more training epochs to become competitive.

# 6 Conclusion

This project implemented the full assignment workflow in one notebook: SentencePiece tokenizer training, manual T5-small construction, span-corruption pre-training, two controlled fine-tuning pipelines, a Qwen zero-shot baseline, and a Qwen retrieval-augmented comparison. The implementation keeps the tokenizer fixed across Pipeline A and Pipeline B, which makes the comparison between pre-training and no pre-training fair. In the final results, Pipeline B clearly outperformed Pipeline A, showing that pre-training on the available Java corpus did not improve downstream bug fixing in this run. The best overall system was Qwen RAG 3-shot, which achieved the highest exact match and CodeBLEU scores on the shared test split. Overall, the experiment shows that retrieval was beneficial, that a larger

prompted model can outperform a smaller task-specific model, and that pre-training must be evaluated empirically rather than assumed to help.

## RepoLinks

- `https://github.com/ABasloe/CSCI555-Assign3/tree/Assign3/Assign3`
- `https://github.com/ABasloe/CSCI555-Assign3/tree/Assign3/Assign3/outputs`